Journal of Manufacturing Systems 61 (2021) 799–807

Contents lists available at ScienceDirect

Journal of Manufacturing Systems

journal homepage: www.elsevier.com/locate/jmansys

Technical Paper

Utilizing uncertainty information in remaining useful life estimation via Bayesian neural networks and Hamiltonian Monte Carlo

Maximilian Benker *, Lukas Furtner, Thomas Semm, Michael F. Zaeh

Institute for Machine Tools and Industrial Management, Technical University of Munich (TUM), Boltzmannstraße 15, 85748 Garching, Germany

ARTICLE INFO

Keywords:
Prognostics and health management
Bayesian neural networks
Remaining useful life
Uncertainty quantification
C-MAPSS

ABSTRACT

The estimation of remaining useful life (RUL) of machinery is a major task in prognostics and health management (PHM). Recently, prognostic performance has been enhanced significantly due to the application of deep learning (DL) models. However, only few authors assess the uncertainty of the applied DL models and therefore can state how *certain* the model is about the predicted RUL values. This is especially critical in applications, in which unplanned failures lead to high costs or even to human harm. Therefore, the determination of the uncertainty associated with the RUL estimate is important for the applicability of DL models in practice. In this article, Bayesian DL models, that naturally quantify uncertainty, were applied to the task of RUL estimation of simulated turbo fan engines. Inference is carried out via *Hamiltonian Monte Carlo* (HMC) and *variational inference* (VI). The experiments show, that the performance of Bayesian DL models is similar and in many cases even beneficial compared to classical DL models. Furthermore, an approach for utilizing the uncertainty information generated by Bayesian DL models is presented. The approach was applied and showed how to further enhance the predictive performance.

1. Introduction

The digital transformation of manufacturing systems, also known as *Industry 4.0*, promises large economic benefits for both, manufacturers and customers. In particular, the implementation of the *internet of things* (IoT) in factories potentially leads to the ability of better tracking the current state of a manufacturing system. This new transparency in combination with flexible manufacturing equipment enables more advanced adaptive planning and control on a factory-level and finally generates economic advantages [1]. One use case, which is especially promising in this context is *condition based maintenance* (CBM) [2,3], which is a strategy, that schedules maintenance actions for machinery based on its estimated condition [4]. As a result, unexpected machine break downs as well as unnecessary early maintenance actions can be avoided. Based on the idea of CBM, the research field *prognostics and health management* (PHM), which combines functions such as condition monitoring, state assessment, failure diagnostics, prognostics and maintenance decision support [5,6], has evolved [7]. Due to the availability of affordable sensor systems, large data sets and algorithmic advances, PHM has been rapidly emerging within the last years [7]. One important facet of PHM is prognostics and more precisely the estimation of *remaining useful life* (RUL) of machinery. The methods applied for estimating RUL can be divided into the following four categories [4]: physics model-based, statistical model-based, *artificial intelligence* (AI) approaches and hybrid approaches, which combine at least two of the former three. Due to advances in the field of AI, the above mentioned AI approaches and in particular *deep learning* (DL) have successfully been applied to the task of RUL estimation [4]. DL models are multilayer neural networks, which can be trained with *back propagation* and *stochastic gradient descent* [8]. One significant advantage of DL is the capability to automatically learn features from raw data and extract patterns, that can enhance the accuracy of the estimated RUL of a machine or component. Although a DL model can outperform other techniques in accurately estimating RUL, the estimate is unlikely to match the actual RUL exactly. Hence, an uncertainty associated with the RUL estimate of a DL model exists. This is critical for decision making. Especially in applications, in which unexpected break downs can lead to high costs or can cause physical harm to humans, the question arises, how reliable, or in other words, how *certain* the RUL estimate is [9]. However, when reviewing the state of the art, it becomes clear, that it is on the one hand still challenging to assess the uncertainty of RUL estimates of DL models and on the other hand it is still difficult to utilize this uncertainty in decision making [4,10,11]. This can be addressed by

* Corresponding author.
E-mail address: maximilian.benker@iwb.tum.de (M. Benker).

https://doi.org/10.1016/j.jmsy.2020.11.005
Received 28 September 2020; Received in revised form 29 October 2020; Accepted 9 November 2020
Available online 7 December 2020
0278-6125/© 2020 The Author(s). Published by Elsevier Ltd on behalf of The Society of Manufacturing Engineers. This is an open access article under the CC BY license (http://creativecommons.org/licenses/by/4.0/).

Abbreviations

AE	auto encoder
AI	artificial intelligence
CBM	condition based maintenance
CNN	convolutional neural networks
DL	deep learning
HMC	Hamiltonian Monte Carlo
IoT	internet of things
LSTM	long short-term memory
MAE	mean absolute error
PHM	prognostics and health management
RBM	restricted Boltzmann machine
RMSE	root mean squared error
RNN	recurrent neural networks
RUL	remaining useful life
VI	variational inference

choosing a Bayesian approach and treating the DL model's parameters as random variables resulting in probabilistic RUL predictions. The resulting advantage is, that the entire distribution of the RUL estimate can be determined instead of only a point estimate. Furthermore, a Bayesian approach to DL naturally provides regularization and is less prone to over fitting [12]. Bayesian DL was, among others, pioneered by Neal [13] in the early 1990s, who successfully conducted inference about the unknown parameters in DL models by applying *Hamiltonian Monte Carlo* (HMC) [14]. However, the success of traditional HMC is dependent on gradients and hyper-parameters, which have to be set by the user and therefore demand a high level of expertise. This hindered a wide-spread application of HMC in Bayesian DL. Subsequently, in the past, practitioners have turned to other ways of conducting inference in Bayesian DL models for RUL estimation instead. Especially approximative inference methods known as *variation inference* (VI) became popular. In this case, the true, complex distribution of the unknown model parameters is approximated with a less complex and known *variational distribution*, from which samples can be drawn and, hence, probabilistic predictions can be derived. Today, however, methodological innovations in HMC and new software allow a feasible application of HMC. Hence, the true distribution of the DL model's parameters can be sampled, which is expected to yield better results and lead to higher accuracies in RUL estimation compared to the approximative approach of VI. Therefore, the objective of this article is the experimental comparison of Bayesian DL models trained with HMC, VI and standard back propagation and gradient descent. The performance of all models is assessed on simulated run-to-failure turbofan engine degradation data. Furthermore, the additional information provided by the Bayesian DL models, namely the uncertainty associated with the RUL estimate, is incorporated in the decision making process. In short, the following novel contributions will be described in this paper:

- Implementation of Bayesian DL models trained with HMC and application to estimate the RUL of simulated turbofan engines.
- Implementation of Bayesian DL models trained with both HMC and VI and comparison of their prediction performance with their non-probabilistic counterpart DL models trained with back propagation and gradient descent.
- A new method for utilizing the Bayesian DL model's uncertainty estimates in decision making is presented.

This article is structured as follows: In Section 2 a brief overview of the state of the art concerning DL models for RUL estimation is given. In Section 3 the theoretical foundations for Bayesian DL models, HMC and VI are given. The proposed approach for utilizing uncertainty information in a decision making process is presented in Section 4. This approach was tested by using run-to-failure data of turbo fan engines and the results are analyzed in Section 5. Finally, the conclusion and outlook are given in Section 6.

2. Related work

Depending on the application domain, DL models with different architectures have been proposed in the literature. For example, *convolutional neural networks* (CNN) are common in computer vision, where detection and classification of objects is a major task [15]. An other example are *recurrent neural networks* (RNN), which can model sequential data and were successfully applied to the domain of speech recognition [16]. In PHM, mostly four types of architectures have been applied: *auto-encoder* (AE), *restricted Boltzmann machine* (RBM), CNN and RNN. The former two architectures were mostly applied to unsupervised learning settings, where no historical failure data is available. In the case of supervised learning, where labels are available, i.e. in the form of historical failure data, mostly CNNs and RNNs were applied [10]. A first attempt to use CNN in the context of PHM was made by Babu et al. [17]. They pre-processed the data with a sliding window approach, which segments a raw, multivariate sensor signal into a collection of shorter signals, supporting the application of convolutional layers. In other words, the sliding window approach transforms the original time series into an image-like representation (see Fig. 1). In addition, the authors conducted so called *label rectification*, which assumes a piece-wise linear degradation model bounding the maximum RUL value (see Fig. 2). The proposed model was trained with back propagation and gradient descent. Finally, they validated their approach by using the popular C-MAPSS data set, which was first published by Saxena et al. in [18]. The results showed superior performance concerning the accuracy of the estimated RUL compared to other AI approaches.

In [19] a novel CNN architecture was introduced. The authors also conducted pre-processing of the raw data by applying a sliding window approach and rectifying the labels. The model was trained with back propagation, gradient descent and *dropout* [20] in order to avoid over fitting. The validation of the model was also conducted on the C-MAPSS

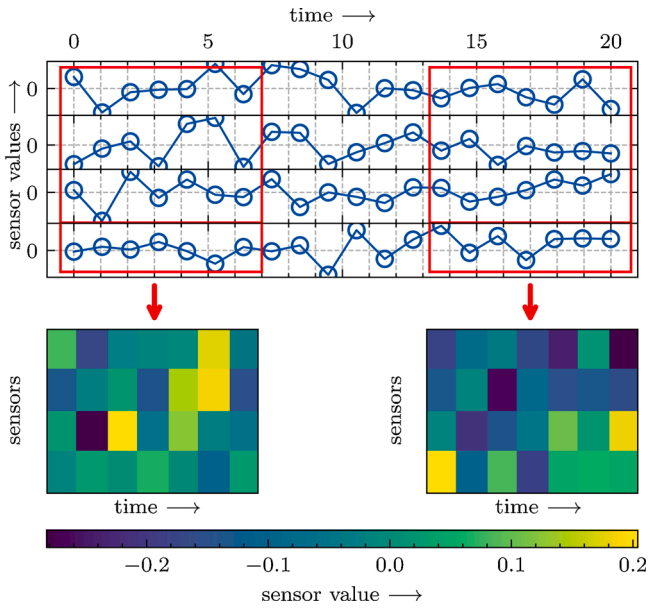


Fig. 1. Exemplary illustration of data pre-processing with a sliding window approach; the original observations of four sensors are transformed into an image observation by taking into account seven subsequent observations at a time. The window of size seven is applied from left to right over the entire time series. The first and the last window is visualized.

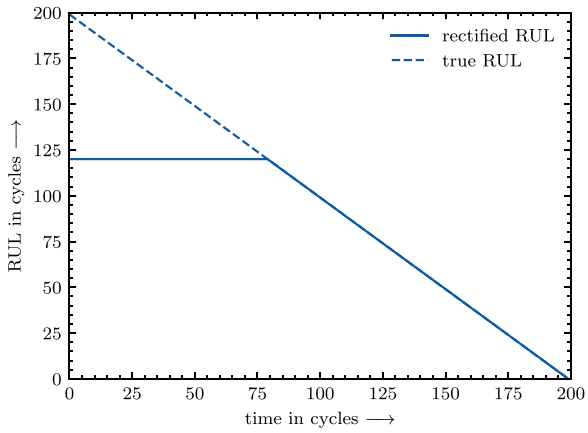


Fig. 2. Exemplary piece-wise linear RUL degradation model limiting the RUL to 120 cycles.

data set. The fitted model achieved high prediction accuracy in RUL estimation compared to a number of different machine learning models, many of which were DL models with different architectures. A combination of a one-layer neural network and a *long short-term memory* (LSTM) DL model was applied in [21]. LSTM is a modified RNN architecture suitable and often applied for extracting features from time series [22]. In a first step, a health indicator was learned by the one-layer neural network. This health indicator summarizes a multivariate time series and ranges from 1 at the beginning of the lifetime to 0 at the time of failure. In a second step, this health indicator was extrapolated with an LSTM DL model. The models were fitted via back propagation and gradient descent. The validation was conducted with the C-MAPSS data set and the authors showed, that their approach can be beneficial compared to others. Another combined approach was presented in [23]. The authors used an RBM along with an LTSM DL model. The RBM was first used to extract abstract features from raw time series data before these features were fed into an LSTM DL model, which predicted the RUL values. They applied label rectification, optimized the model's hyper-parameters with a genetic algorithm and fitted the model via back propagation and gradient descent. The validation was carried out with the C-MAPSS data set and the authors reported superior performance compared to the state of the art. Although all of the above mentioned articles present successful DL models for RUL estimation, none of them assesses the uncertainty associated with the estimates. Only recently, several articles quantifying this uncertainty in DL models for RUL estimation were published. In [24] a combined, probabilistic approach was presented consisting of three additive model parts: An explicit lifetime model only dependent on time t, a linear model dependent on sensor signals at time t and an RNN dependent on both time t and the entire sequence of sensor signals up to time t. One advantage of this model formulation is the possibility of combining interpretable, simpler models with a flexible but less interpretable RNN. Inference about all parameters was conducted with VI and yielded significant improvements over classical AI approaches.

Another Bayesian DL model was applied in [25]. The authors used the model for mapping the input sensor signals to parameters of a lifetime distribution. Since over time, and hence degradation, the sensor signals change, the lifetime distribution's parameters and subsequently the distribution of the RUL also changes. An additional advantage of this approach is the ability to separate aleatoric uncertainty, which stems from the model's parameters, and epistemic uncertainty, which originates from random errors such as sensor noise. The latter is reflected in the lifetime distribution. The model parameters were estimated via VI and the experiments were conducted with different lifetime distributions such as the Weibull distribution. The model was validated on a proprietary data set of high-voltage circuit breaker run-to-failure data.

A comparison between non-probabilistic DL models trained with back propagation and gradient descent and Bayesian DL models was conducted in [26]. For RUL estimation the authors proposed a Bayesian multi-scale CNN model which was validated with a ball bearing degradation data set and a Bayesian LSTM model which was validated with the C-MAPSS data set. Both models' performances were compared to their non-probabilistic counterparts and were trained with VI. In the case of the C-MAPSS data set only one of the four available subsets was used and a sliding window approach was applied. Although the performance was comparable to the state of the art, the uncertainty information was not utilized in the proposed approach.

Summarizing, many contributors in the literature successfully used DL models to predict the RUL of machinery. In order to be able to compare different approaches, the C-MAPSS data set has established itself as a popular benchmark data set for validating DL models for RUL estimation. The sliding window and label rectification pre-processing steps are commonly adopted for this data set. However, it is notable, that other (often proprietary) data sets have successfully been applied as well [27–30]. Recently, Bayesian DL models were successfully presented, often outperforming other, non-probabilistic models. However, to the best of the authors knowledge, there has not been an approach yet, that trained Bayesian DL models with HMC in the context of RUL estimation, which conducted comparisons with non-probabilistic DL models and presented an end-to-end method including the utilization of the additional uncertainty information.

3. Theoretical background

3.1. Bayesian neural networks

Following a Bayesian approach, all uncertain (i.e. unknown) parameters are treated as random variables, which are inferred from data by determining the so called *posterior distribution*. Formally, the posterior distribution $p(\theta|D,f)$ of the unknown parameters θ given the data $D = \{(\mathrm{x}_i, y_i)\}_{i=1}^{N}$ and the model f is defined as [31]

$$p(\theta|D,f) = \frac{p(D|\theta,f)p(\theta|f)}{p(D|f)}, \tag{1}$$

where $p(D|\theta,f)$ is the *likelihood* of the parameters θ, $p(\theta|f)$ is the *prior* probability of the parameters θ and $p(D|f)$ is the *marginal likelihood* defined as

$$p(D|f) = \int_{\theta} p(D|\theta,f)p(\theta|f)d\theta. \tag{2}$$

Knowing the posterior distribution from Eq. (1) for a given model f, one can construct the *posterior predictive distribution* for a new output $y^{\star}$ based on a new input $\mathrm{x}^{\star}$ by computing [31]

$$p(y^{\star}|\mathrm{x}^{\star},D,f) = \int_{\theta} p(y^{\star}|\mathrm{x}^{\star},\theta,f)p(\theta|D,f)d\theta. \tag{3}$$

In the case, where the model f is defined as a certain neural network, i.e. $y^{\star} = f(\mathrm{x}^{\star},\theta)$, the predictive distribution is reduced to [13]

$$p(y^{\star}|\mathrm{x}^{\star},D) = \int_{\theta} f(\mathrm{x}^{\star},\theta)p(\theta|D)d\theta. \tag{4}$$

In practice, predictions can be derived by first sampling parameters $\theta^{(m)}$ from the posterior $p(\theta|D)$, then computing the model predictions $f(\mathrm{x}^{\star}, \theta^{(m)})$ and finally calculating statistical moments from the finite sample of model predictions. For example, the expected value for a new outcome can be calculated as follows [13]:

$$\mathbb{E}[y^{\star}] \approx \frac{1}{M}\sum_{m=1}^{M} f(\mathrm{x}^{\star},\theta^{(m)}) \tag{5}$$

Unfortunately, in practice, the marginal likelihood in Eq. (2) and hence the posterior distribution in Eq. (1) is rarely tractable and often cannot be computed analytically [31]. Fortunately, approximations for the posterior distribution have been developed, two of which will be presented in the following sections.

3.2. Hamiltonian Monte Carlo

HMC is one variant of a whole class of algorithms named *Markov chain Monte Carlo* (MCMC). The idea behind the application of MCMC is to approximate the posterior distribution $p(\theta|D,f)$ defined in Eq. (1) by constructing a Markov chain, whose stationary distribution matches the desired posterior distribution. If this can be achieved, one can sample from the Markov chain and compute quantities of interest such as the expected value for a new outcome defined in Eq. (5) [32]. Intuitively, MCMC methods start with a set of unknown parameters θ and move to a new point $\tilde{\theta}$, which is proposed by the MCMC algorithm. The proposed parameters $\tilde{\theta}$ are accepted depending on the resulting model fit to the training data. In case the proposed parameters are accepted, $\tilde{\theta}$ is set to be a new observation of the unknown parameters θ and becomes part of the Markov chain. However, classical MCMC algorithms, such as [33,34], can be slow in high dimensional spaces. That is due to the fact, that those classical MCMC algorithms tend to explore high dimensional spaces with a random walk behaviour and, hence, rarely propose new parameters $\tilde{\theta}$ that are in regions of high probability [32]. Subsequently, in the case of DL modes with thousands or even millions of parameters, and therefore high-dimensional parameter spaces, the application of those classical approaches becomes impractical [13]. Instead, HMC can be applied; a method, which is less prone to random walk behaviour in exploring high dimensional spaces due to better proposals $\tilde{\theta}$ achieved by exploiting gradient and geometry information about the posterior distribution and therefore producing less rejection rates and faster convergence [32]. In the following, the main technical implications of HMC based on [35] are briefly outlined. The HMC algorithm originated from work in the area of lattice field theory [36]. In light of this, a sample of the unknown parameters is equivalent to a position of a particle. Since every particle is also described with a momentum variable r, this is adopted to the unknown parameters θ as well. Subsequently, instead of only modelling the distribution of the parameters θ, the joint distribution over unknown parameters θ and their momentum r is modelled as [32]

$$p(\theta, r) = exp\{-H(\theta, r)\}, \tag{6}$$

where $H(\theta, r) = -log[p(\theta, r)] = -log[p(\theta|r)p(r)]$ is the so called *Hamiltonian* function. In the case, where the momentum variables r are sampled from a Gaussian distribution, Eq. (6) is reduced to the following unnormalized joint distribution[1] [37]:

$$p(\theta, r) \propto exp\left\{\mathcal{L}(\theta) - \frac{1}{2} r \cdot r\right\}, \tag{7}$$

with $\mathcal{L}(\theta) = log[p(y|x,\theta)p(\theta)]$ representing the likelihood of the data. After initializing the momentum $r^0 \sim \mathcal{N}(0, I)$, new sampled parameters $\tilde{\theta}$ are accepted with probability α [37]:

$$\alpha = min\left\{1, \frac{exp\left\{\mathcal{L}(\tilde{\theta}) - \frac{1}{2}\tilde{\tilde{r}}\cdot\tilde{\tilde{r}}\right\}}{exp\left\{\mathcal{L}(\theta) - \frac{1}{2} r^0 \cdot r^0\right\}}\right\}, \tag{8}$$

where the so called *leapfrog* steps [37] in Eqs. (9)–(11) have to be computed L times with stepsize ε:

$$\tilde{r} = r + \frac{\varepsilon}{2}\nabla_\theta \mathcal{L}(\theta), \tag{9}$$

$$\tilde{\theta} = \theta + \varepsilon \tilde{r}, \tag{10}$$

$$\tilde{\tilde{r}} = r + \frac{\varepsilon}{2}\nabla_\theta \mathcal{L}(\tilde{\theta}). \tag{11}$$

The choice of the two hyper-parameters L and ε is critical for the success of classical HMC algorithms. They are usually hard to set. However, algorithmic advances such as the ones presented by Hoffman and Gelman [37] can automatically tune those hyper-parameters.

3.3. Variational inference

Rather than sampling from a Markov chain, VI tries to approximate the posterior distribution $p(\theta|D,f)$ from Eq. (1) by optimizing the parameters Θ of a simpler, known probability distribution $q(\theta|\Theta)$, such that the *Kullback-Leibler* (KL) divergence with the true posterior is minimized [38,39]:

$$\begin{aligned}\Theta^\star &= \underset{\Theta}{arg\,min}\,\mathrm{KL}[q(\theta|\Theta)||p(\theta|D,f)],\\ &= \underset{\Theta}{arg\,min}\int_\theta q(\theta|\Theta) log\frac{q(\theta|\Theta)}{p(D|\theta,f)p(\theta|f)}d\theta,\\ &= \underset{\Theta}{arg\,min}\mathrm{KL}[q(\theta|\Theta)||p(\theta|f)] - \mathbb{E}_{q(\theta|\Theta)}[logp(D|\theta,f)].\end{aligned} \tag{12}$$

The first part of the solution in Eq. (12) can be interpreted as a complexity cost, whereas the second part of the equation represents the likelihood cost. Therefore, natural regularization is built into the variational approach. The cost function, which is sometimes called *expected lower bound* [39], can be re-written as

$$\mathcal{F}(D,\Theta) = \mathrm{KL}[q(\theta|\Theta)||p(\theta|f)] - \mathbb{E}_{q(\theta|\Theta)}[logp(D|\theta,f)]. \tag{13}$$

The key difference compared to HMC – framing the problem of finding $p(\theta|D,f)$ as an optimization problem – has the advantage of significantly faster inference and better scalability to large data sets. Nevertheless, there is no guarantee for coming close to the true posterior distribution $p(\theta|D,f)$ [38]. For the special case of DL models, an algorithm named *Bayes by Backprop* was presented by [39], which approximates the expected lower bound from Eq. (13) by

$$\begin{aligned}\mathcal{F}(D,\Theta) \approx \sum_{m=1}^{M} logq(\theta^{(m)}|\Theta) \quad &-logp(\theta^{(m)}|f)\\ &-logp(D|\theta^{(m)}),\end{aligned} \tag{14}$$

where $\theta^{(m)}$ denotes the mth Monte Carlo sample drawn from $q(\theta^{(m)}|\Theta)$. In practice, gradient based methods can be applied to minimize the approximated expected lower bound in Eq. (14) [39].

4. Proposed approach

The main objectives of this paper, as outlined in Section 1, require the implementation and comparison of different DL models for RUL estimation. Furthermore, the uncertainty information received from Bayesian DL models was intended to be utilized in decision making. The following sections describe how the two aspects are considered in detail.

4.1. Deep learning models

In order to compare non-probabilistic and Bayesian DL models, eight different ones were implemented, applied and analyzed using a simulated data set. The models have the following two architectures:

- D3: This is a simple baseline model. The input matrix is flattened into a one-dimensional vector and fed to three consecutive, fully

[1] $r \cdot r$ being the inner product of vector r with itself.

connected layers with 100 neurons each and a sigmoid activation function at the output of each layer. A final output neuron returns the estimated RUL.

- C2P2: This is a CNN model, originally proposed by [17]. The network consists of two subsequent convolutional and pooling layers. The first 5×14 convolutional layer with 8 channels is followed by a 2×1 average pooling layer. The second 2×1 convolutional layer with 14 channels is also followed by a 2×1 average pooling layer. All convolutional layers are followed by a sigmoid activation function. The output of the last convolutional layer is flattened and fed into a single, fully connected output neuron, that returns the predicted RUL.

The two architectures described above were implemented and fitted with the following four inference algorithms:

- Backpropagation & stochastic gradient descent (BP): Classic neural network training with backpropagation, stochastic gradient descent and a *mean squared error* (MSE) cost function was conducted. The *Adam* optimizer [40] with a learning rate of 0.001 was applied for 200 epochs of training. For fine-tuning, at the end of learning, the learning rate was set to 0.0001 for another 50 epochs. Note, that this method finds a single set of parameters θ and does not produce a posterior distribution of parameters $p(\theta|D,f)$. Hence, the predictions are point estimates.
- Hamiltonian Monte Carlo (H1): Probabilistic inference with an HMC sampler, provided by the open source software *hamiltorch*[2], for learning the posterior distribution of the unknown model parameters θ was conducted. The initial step size ε was set to 0.0012 and the number of leapfrog steps L was set to 10. The step size was automatically optimized in further training. The prior over the unknown parameters $p(\theta)$ was set to be Gaussian with randomly sampled mean values and variance $\sigma_\theta^2 = 100$. The variance was set to be rather large, since this enables the parameters to explore a large parameter space. The noise variance of the data σ_N^2, which is a hyper-parameter of the HMC implementation in *hamiltorch*[2], was set to the total variance of the RUL labels in the training data set. The number of samples was set to 500 and the first 250 samples are burned, i.e. not considered for inference.
- Hamiltonian Monte Carlo pre-trained (H2): As in H1, probabilistic inference with an HMC sampler, provided by the open source software *hamiltorch*[2] was conducted. The hyper-parameters were set equal to those of the H1 inference. However, the mean of the prior distribution of the unknown parameters $p(\theta)$ was not initialized randomly as in the inference scheme H1 described above, but the results of BP inference were used as initialization values. Since the unknown parameters were already pre-trained, the number of samples was set to 50 and only the first 25 samples were burned.
- Variational inference (VI): Probabilistic inference with a VI scheme provided by the open source software package *BLiTZ*[3] was conducted. The Bayes by Backprop algorithm, presented in Section 3.3, was applied for 250 epochs. The Adam optimizer with a MSE error function and a learning rate of 0.001, which was decreased to 0.0001 after 200 epochs, was used. In order to approximate the loss function, 10 samples were drawn from the approximated posterior distribution. The standard deviation of the prior distribution of the unknown parameters $p(\theta)$ was set to $\sigma = log(1 + exp(\rho))$, with $\rho = -1$.

Table 1
Overview and names of all applied DL models.

Model architecture	Inference	Name
D3	BP	D3_BP
	H1	D3_H1
	H2	D3_H2
	VI	D3_VI
C2P2	BP	C2P2_BP
	H1	C2P2_H1
	H2	C2P2_H2
	VI	C2P2_VI

The models are summarized and named in Table 1 for clarity. In the case of Bayesian DL models, the entire distribution of the predicted RUL was calculated. Hence, an uncertainty can be associated with the RUL. In the next section, a new approach to utilizing this uncertainty in decision making is proposed.

4.2. Utilizing uncertainty information

In case of the Bayesian DL models, the estimated RUL is a probability distribution. Hence, the variance, i.e. the uncertainty of the RUL estimate, is quantified and can be utilized. In the following, an approach is presented how this can be done.

There are two underlying assumptions: First, the deviation between the predicted RUL $y^\star$ and the true RUL y is defined as $\tau = y^\star - y$. It is further assumed, that τ follows a Gaussian distribution, i.e. $p(\tau|\mu_\tau, \sigma_\tau) = \mathcal{N}(\tau|0, \sigma_\tau)$. Note, that consequently the expected value of the estimated RUL $y^\star$ is assumed to be equal to the true RUL y. This distribution can be re-parametrized with $\mu_\tau = b\sigma_\tau$ as

$$p(\tau|b\sigma_\tau, \sigma_\tau) = \frac{1}{\sqrt{2\pi\sigma_\tau^2}} exp\left[-\frac{(\tau - b\sigma_\tau)^2}{2\sigma_\tau^2}\right] \tag{15}$$

In case $b = 0$, then $p(\tau|\mu_\tau, \sigma_\tau) = p(\tau|b\sigma_\tau, \sigma_\tau)$. The second assumption is that a user's cost function is given by the so called score function $s(\tau)$ defined as [18]

$$s(\tau) = \begin{cases} s_1(\tau) = e^{-\frac{\tau}{13}} - 1, & \text{for } \tau < 0. \\ s_2(\tau) = e^{\frac{\tau}{10}} - 1, & \text{for } \tau \geq 0. \end{cases} \tag{16}$$

This cost function implicitly assumes that unexpected failures are more costly than unnecessarily early maintenance actions. The total costs can be calculated by the multiplication of the score function $s(\tau)$ with $p(\tau|0, \sigma_\tau)$ and integrating over τ. Since the cost function is asymmetric, the total cost c_{total} can be reduced by shifting the distribution of the estimated RUL in negative τ direction. This is equivalent to decreasing the mean value of $p(\tau|b\sigma_\tau, \sigma_\tau)$ by a factor of b (see Fig. 3). Hence, the objective is to find a certain $\tilde{b}$, that minimizes the total costs:

$$\widetilde{b} = \underset{b}{argmin}(c_{\text{total}}) \quad = \underset{b}{argmin}\left(\int_{-\infty}^{+\infty} p(\tau|b\sigma_\tau, \sigma_\tau)s(\tau)d\tau\right) \quad = \underset{b}{argmin}\left(\int_{-\infty}^{0} p(\tau|b\sigma_\tau, \sigma_\tau)s_1(\tau)d\tau + \int_{0}^{+\infty} p(\tau|b\sigma_\tau, \sigma_\tau)s_2(\tau)d\tau\right) \tag{17}$$

Note, that the only quantity needed to compute Eq. (17) is the standard deviation of the estimated RUL. Fortunately, this is available from the

[2] https://github.com/AdamCobb/hamiltorch
[3] https://github.com/piEsposito/blitz-bayesian-deep-learning

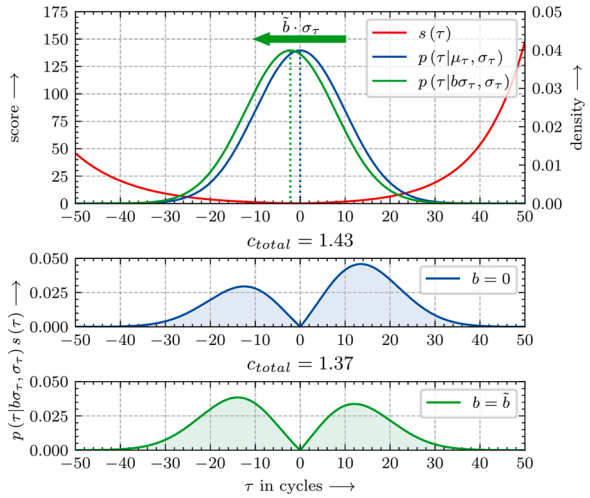


Fig. 3. Utilization of the uncertainty of the RUL estimate. In the upper sub-figure the mean of the RUL distribution is shifted by the factor $\tilde{b}$, leading to different costs $p(\tau|b\sigma_\tau, \sigma_\tau)s(\tau)$. This can be seen in the lower two sub-figures, where the blue curve represents the original cost distribution and the green curve represents the enhanced cost distribution. The area under the curves represents the total costs c_{total}. The total costs for $b = \tilde{b}$ are lower due to the asymmetric nature of the scoring function $s(\tau)$. (For interpretation of the references to colour in this figure legend, the reader is referred to the web version of this article.)

Bayesian DL models' predictions. Solving this optimization problem is non-trivial and there are several approaches to do so. In this article the problem was solved for different values of σ_τ with the computer algebra systems *SymPy*[4] and *Geogebra*[5] by taking the derivative of Eq. (17) with respect to b, setting the derivative to zero and solving for b.

5. Experiments

5.1. Data set

As already stated in Section 2, the C-MAPSS data was originally published by Saxena et al. [18] with the intent to provide data for the challenge of RUL estimation. The data set consists of simulated run-to-failure data of turbo fan engines. The data set is divided into four subsets, each differing in the number of fault modes, the number of operating modes and the amount of training data (see table 2). Each subset consists of training data, which contains the entire run-to-failure sequence, and test data, which contains a sequence of values that stops prior to the failure. At the time the sequence stops the RUL is given and has to be estimated as accurately as possible by the proposed model. Each sequence consists of 21 equally spaced sensor time series. In the first subset, only one fault mode and operating condition is recorded. In

Table 2
Overview of the different C-MAPSS subsets.

	Subset			
	FD001	FD002	FD003	FD004
Number of training data sequences	100	260	100	249
Number of test data sequences	100	259	100	248
Operating conditions	1	6	1	6
Fault modes	1	1	2	2

[4] https://github.com/sympy/sympy
[5] https://www.geogebra.org

Table 3
Summary of pre-processed training and test data.

	Subset			
	FD001	FD002	FD003	FD004
Length of pre-processed training data	17,731	48,819	21,820	57,763
Length of pre-processed test data	100	259	100	248
Number of selected features	14	24	14	24
Time window length L_w	30	20	30	15

the other three subsets, up to six different operating conditions and up to two fault modes are simulated and recorded (see table 2). However, the implications of the operating conditions and fault modes are not known.

5.2. Data pre-processing

The data was pre-processed with label rectification, feature (i.e. sensors and operating conditions) selection, normalization and sliding window application. In accordance with [19], the maximum RUL was set to 125. For the subsets FD001 and FD003, the sensors 1, 5, 6, 10, 16, 18 and 19 provided little or no information regarding the health status of a turbo fan engine. Thus they are excluded from the two subsets. Furthermore, the data was normalized with a min-max normalization. Formally, every sensor value $s_{i,t}$ is transformed to

$$s_{i,t}^{\text{norm}} = \frac{2(s_{i,t} - s_{i,\min})}{s_{i,\max} - s_{i,\min}} - 1, \tag{18}$$

with i being the index of the sensor, t being time in cycles and $s_{i,\min}$ and $s_{i,\max}$ being the minimum and maximum values of the sensor sequence, respectively. The sliding window approach was applied with a window of length L_w, differing from subset to subset (see Table 3). Since the window slides over the entire sequence of features of length T, $T - L_w + 1$ matrices of shape $L_w \times d$ are generated, where d is the dimensionality of the multivariate feature sequence, i.e. the number of selected sensors and operating conditions. Those matrices were the inputs to the DL models described in section 4.1. The associated RUL values to the matrices were the labels (i.e. outputs) of the models.

5.3. Assessing the remaining useful life prediction accuracy

In order to be able to compare the presented approach with the results in the literature, several performance metrics were applied, which will be presented in the following. The quantity of interest is again the deviation between the predicted RUL $y^\star$ and the true RUL y, which is defined as $\tau = y^\star - y$ (see Section 4.2). In case of Bayesian DL models, in which the entire distribution of $y^\star$ is calculated, the mean value of the Monte Carlo samples was used as RUL prediction. For all metrics, K denotes the length of the test data set. The applied metrics include the *mean absolute error* (MAE), the *root mean square error* (RMSE), defined in Eqs. (19) and (20) respectively, and the score function defined in Eq. (16):

$$\text{MAE} = \frac{1}{K}\sum_{k=1}^{K} |\tau_k|. \tag{19}$$

$$\text{RMSE} = \sqrt{\frac{1}{K}\sum_{k=1}^{K} \tau_k^2}. \tag{20}$$

The score function was evaluated and reported in two ways:

- Score1: The score function is calculated based on the prediction of each model.
- Score2: The approach for utilizing the uncertainty of the Bayesian DL models presented in Section 4.2 was applied to the RUL estimate $y^\star$.

Hence, after drawing the Monte Carlo samples from the Bayesian DL models to estimate $y^\star$, the variance σ_τ was calculated. Based on this variance and Eq. (17), $\tilde{b}$ was obtained and finally the enhanced RUL prediction $y^\star_{\text{new}} = y^\star + \tilde{b}\sigma_\tau$ was determined. Based on the enhanced RUL prediction $y^\star_{\text{new}}$ the score was calculated and reported.

5.4. Experimental results and discussion

All model architectures presented in Section 4.1 were implemented in *PyTorch*[6] along with the above mentioned, additional Bayesian inference software packages. All computations were conducted on a *Leibniz Supercomputing Centre*[7] compute node virtual machine with 20 Intel® Xeon® Gold 6148 vCPUs, one Nvidia® Tesla V100, 368 GB RAM, *PyTorch* v.1.4.0 and CUDA 10.1. Exemplary predictive distributions of the D3 model architecture with different inference methods are shown in Fig. 4. There, it can be seen that they yield different results for both, the mean estimated RUL and the resulting predictive distribution. In the shown example, the H1 and VI inference methods produce comparable densities with similar mean values and relatively large uncertainty estimates. The H2 inference method, which is initialized with the BP model's parameters, produces a more narrow density and a less accurate prediction. This is in line with expectations, since the H2 method was initialized with the parameters learned by the BP inference method and therefore is biased towards these parameters. The predictive accuracies with respect to the performance metrics defined in Section 5.3 are given in Tables 4 and 5for the D3 and C2P2 model architectures respectively. In general, the predictive performance differs significantly between the different subsets for all models. On the first and third subsets (FD001 and FD003) all models tend to perform better compared to the second (FD002) and fourth (FD004) subset. The drop in performance was expected, since the additional fault modes and operating conditions make it harder to detect degradation patterns in the data. However, since the performance on FD003 is better than the one on FD002 for most metrics, the different fault modes are detected more easily by the models than the different operational conditions in subsets FD002 and FD004. For these, the performance of all models is the worst in all experiments. This is indeed the case for both, the Bayesian and the non-probabilistic models. When comparing the architectures, the difference between the best D3 and C2P2 models for all subsets is small, although in the conducted experiments, the best D3 models slightly outperform the best C2P2 model in almost all cases. Hence, the simple fully connected model can already achieve good results and even outperform the more complicated CNN. This is especially important to be taken into account when computing resources are limited. When comparing the different inference approaches, it can be stated, that for the RMSE and MAE the Bayesian DL models show comparative performances to their non-probabilistic counterparts in all experiments, sometimes even outperforming them. More interesting, when looking at the score1 values, the Bayesian models outperform their counterparts in all experiments. This has been achieved already without utilizing uncertainty information. One possible explanation for this effect is the natural regularization of Bayesian approaches. Hence, choosing a Bayesian approach to a given model architecture alone promises to enhance the predictive accuracy instantly without considerable additional effort. When comparing the different Bayesian inference approaches, HMC and VI yield both good results. However, the performance of the two depends on the model architectures. For the simple but well performing baseline architecture D3, the VI inference method yields slightly better results than the HMC inference methods. The differences are rather small, however. In case of the more complex architecture C2P2, the HMC inference method with pre-trained parameters, H2, yields significantly better results than H1 and VI. Hence, in more complex models, pre-training is recommended. When assessing the proposed method for utilizing the uncertainty, the experiments confirm the expected improvement in performance. All calculated score2 values are better than the initial score1 values. Hence, in the case where the cost function is defined by the score function, the proposed approach yields good results. Note, that this is the case even though the original assumptions stated in Section 4.2 are not always met, as can be seen in Fig. 4. There, the distributions are not perfectly Gaussian and the expected values of the estimated RUL $y^\star$ are not perfectly equal to the true RUL y. This becomes especially visible in the predictive distribution of D3_H2. Nevertheless, the results are superior to the calculated score1, which speaks for the robustness of the proposed approach. Summarizing, it can be stated, that, in accordance with the related work presented in Section 2, the application of Bayesian DL models is advantageous in the domain of RUL estimation. Especially when utilizing the uncertainty information, the full potential of Bayesian DL models is exploited, which was successfully presented in this section.

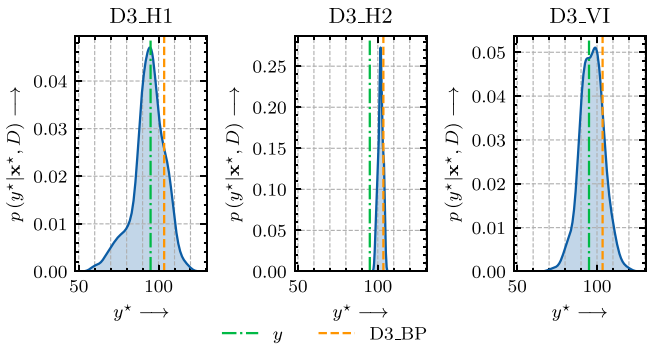


Fig. 4. Exemplary predictive distributions of the estimated RUL $y^\star$ of the Bayesian D3 models for turbo fan number eight in the test data of subset FD001; the non-probabilistic model D3_BP, which only produces point estimations, and the true RUL y are indicated by vertical lines.

6. Conclusion and future work

Within the work described in this article, Bayesian DL models were applied to the problem of RUL estimation for simulated turbo fan engines. Those models were successfully implemented and trained with HMC and VI. The models were compared to their non-probabilistic counterparts fitted with backpropagation and gradient descent. The results show, that the application of Bayesian DL models is beneficial in almost all cases without significant additional costs, since modern software enables practitioners to benefit from latest algorithmic advances in Bayesian inference. Furthermore, an approach for utilizing the uncertainty information provided by Bayesian DL models was presented and applied, which further improved the results. Hence, the quantification and utilization of uncertainties in RUL estimation via Bayesian DL models is recommended. Future research in utilizing the uncertainty information from Bayesian DL models should focus on the following topics: First, the estimated uncertainties are not always calibrated (see Fig. 4). However, methods have been proposed to fix this problem [41]. Second, other model architectures presented in the literature such as LSTM and AE should be investigated and treated as Bayesian DL models to further improve the predictive accuracy. Third, the method for utilizing uncertainty presented in Section 4 should be extended to also handle non-Gaussian densities for τ. Fourth, since different use cases are likely to exhibit different cost functions from the one presented in Eq. (16), the investigation of others should be pursued.

[6] https://pytorch.org
[7] https://www.lrz.de/services/compute/cloud_en

Table 4
RMSE, MAE, Score1 and Score2 of all D3 models for the test data with rectified labels; the mean and standard deviation were calculated from 10 runs with different random seeds. The best results are marked in bold characters.

Subset	Metric	D3_BP		D3_H1		D3_H2		D3_VI	
		Mean	Std	Mean	Std	Mean	Std	Mean	Std
FD001	RMSE	**13.84**	0.25	14.63	1.01	13.98	0.57	14.01	0.10
	MAE	**9.69**	0.22	11.07	1.49	9.86	0.53	10.07	0.03
	Score1 $\cdot 10^{-2}$	4.27	0.47	4.12	0.39	4.39	0.47	**3.93**	0.15
	Score2 $\cdot 10^{-2}$	–	–	3.71	0.33	4.31	0.46	**3.64**	0.13
FD002	RMSE	20.74	0.99	37.99	4.86	20.69	0.92	**19.40**	0.22
	MAE	**14.83**	0.70	33.14	4.04	14.86	0.67	15.19	0.19
	Score1 $\cdot 10^{-2}$	194.00	143.64	370.29	230.67	204.07	150.50	**28.98**	2.74
	Score2 $\cdot 10^{-2}$	–	–	336.46	231.58	201.20	149.81	**25.52**	2.26
FD003	RMSE	14.41	6.00	16.11	1.69	**14.18**	6.06	14.90	0.27
	MAE	**10.05**	4.55	12.68	1.72	**10.05**	4.61	10.88	0.18
	Score1 $\cdot 10^{-2}$	29.77	83.47	5.71	1.69	29.02	81.80	**5.69**	0.58
	Score2 $\cdot 10^{-2}$	–	–	**4.80**	1.21	28.82	81.45	5.07	0.53
FD004	RMSE	**22.73**	0.74	39.53	3.80	22.76	0.58	23.10	0.14
	MAE	**16.46**	0.39	33.88	3.03	16.58	0.34	17.92	0.10
	Score1 $\cdot 10^{-2}$	103.76	65.21	621.22	322.91	98.17	65.03	**66.95**	5.57
	Score2 $\cdot 10^{-2}$	–	–	546.63	327.44	94.84	61.79	**55.26**	4.72

Table 5
RMSE, MAE, Score1 and Score2 of all C2P2 models for the test data with rectified labels; the mean and standard deviation were calculated from 10 runs with different random seeds. The best results are marked in bold characters.

Subset	Metric	C2P2_BP		C2P2_H1		C2P2_H2		C2P2_VI	
		MEAN	Std	Mean	Std	Mean	Std	Mean	Std
FD001	RMSE	17.86	0.04	20.84	1.57	**17.76**	0.07	21.80	0.15
	MAE	13.29	0.04	15.15	1.20	**13.22**	0.07	15.45	0.12
	Score1 $\cdot 10^{-2}$	7.09	0.10	27.28	13.78	**6.98**	0.17	36.78	2.21
	Score2 $\cdot 10^{-2}$	–	–	24.44	13.20	**6.92**	0.16	33.64	2.05
FD002	RMSE	**19.32**	0.21	32.55	4.91	19.35	0.23	20.14	0.30
	MAE	**15.25**	0.20	28.21	4.59	15.30	0.21	15.99	0.32
	Score1 $\cdot 10^{-2}$	25.29	1.40	193.12	104.54	**25.23**	1.63	29.75	2.14
	Score2 $\cdot 10^{-2}$	–	–	178.37	98.50	**25.00**	1.60	28.37	1.96
FD003	RMSE	20.51	0.10	25.73	1.74	**20.39**	0.09	22.34	0.18
	MAE	15.18	0.04	21.65	1.90	**15.12**	0.06	16.44	0.13
	Score1 $\cdot 10^{-2}$	16.67	0.69	18.55	0.98	**16.23**	0.51	27.82	1.21
	Score2 $\cdot 10^{-2}$	–	–	**14.07**	1.57	16.05	0.51	25.52	1.02
FD004	RMSE	22.38	0.26	36.71	4.05	**22.37**	0.26	24.09	0.33
	MAE	**17.12**	0.32	31.22	3.70	17.13	0.32	18.73	0.30
	Score1 $\cdot 10^{-2}$	59.76	9.27	453.06	272.56	**58.33**	8.92	72.41	9.18
	Score2 $\cdot 10^{-2}$	–	–	407.77	228.94	**57.72**	8.88	69.02	8.49

Author contributions

Maximilian Benker: Conceptualization, methodology, validation, investigation, software, writing – original draft, supervision. Lukas Furtner: Methodology, validation, investigation, software, writing – review & editing. Thomas Semm: Writing – review & editing, supervision, funding acquisition. Michael F. Zaeh: Writing – review & editing, supervision, funding acquisition.

Conflict of interest

The authors declare that there is no conflict of interest.

Declaration of Competing Interest

The authors report no declarations of interest.

Acknowledgments

This project has received funding from the European Union's Horizon 2020 Research and Innovation Programme under grant agreement no. 768575.

References

[1] Mourtzis D, Vlachou E. A cloud-based cyber-physical system for adaptive shop-floor scheduling and condition-based maintenance. J Manuf Syst 2018;47:179–98. https://doi.org/10.1016/j.jmsy.2018.05.008.

[2] Gao R, Wang L, Teti R, Dornfeld D, Kumara S, Mori M, et al. Cloud-enabled prognosis for manufacturing. CIRP Ann 2015;64(2):749–72. https://doi.org/10.1016/j.cirp.2015.05.011.

[3] Montero Jimenez JJ, Schwartz S, Vingerhoeds R, Grabot B, Salaün M. Towards multi-model approaches to predictive maintenance: a systematic literature survey on diagnostics and prognostics. J Manuf Syst 2020;56:539–57. https://doi.org/10.1016/j.jmsy.2020.07.008.

[4] Lei Y, Li N, Guo L, Li N, Yan T, Lin J. Machinery health prognostics: a systematic review from data acquisition to RUL prediction. Mech Syst Signal Process 2018;104:799–834. https://doi.org/10.1016/j.ymssp.2017.11.016.

[5] Sheppard J, Kaufman M, Wilmer T. IEEE standards for prognostics and health management. IEEE Aerosp Electron Syst Mag 2009;24(9):34–41. https://doi.org/10.1109/MAES.2009.5282287.

[6] Adams S, Malinowski M, Heddy G, Choo B, Beling PA. The WEAR methodology for prognostics and health management implementation in manufacturing. J Manuf Syst 2017;45:82–96. https://doi.org/10.1016/j.jmsy.2017.07.002.

[7] Lee J, Wu F, Zhao W, Ghaffari M, Liao L, Siegel D. Prognostics and health management design for rotary machinery systems – reviews, methodology and applications. Mech Syst Signal Process 2014;42(1–2):314–34. https://doi.org/10.1016/j.ymssp.2013.06.004.

[8] LeCun Y, Bengio Y, Hinton G. Deep learning. Nature 2015;521(7553):436–44. https://doi.org/10.1038/nature14539.

[9] Sankararaman S. Significance, interpretation, and quantification of uncertainty in prognostics and remaining useful life prediction. Mech Syst Signal Process 2015;52-53:228–47. https://doi.org/10.1016/j.ymssp.2014.05.029.

[10] Zhang L, Lin J, Liu B, Zhang Z, Yan X, Wei M. A review on deep learning applications in prognostics and health management. IEEE Access 2019;7:162415–38. https://doi.org/10.1109/ACCESS.2019.2950985.

[11] Fink O, Wang Q, Svensén M, Dersin P, Lee W-J, Ducoffe M. Potential, challenges and future directions for deep learning in prognostics and health management applications. Eng Appl Artif Intell 2020;92:103678. https://doi.org/10.1016/j.engappai.2020.103678.

[12] Ghahramani Z. Probabilistic machine learning and artificial intelligence. Nature 2015;521(7553):452–9. https://doi.org/10.1038/nature14541.

[13] Neal RM. Bayesian learning for neural networks. Lecture notes in statistics, vol. 118. New York: Springer; 1996.

[14] Betancourt M. The convergence of Markov chain Monte Carlo methods: from the Metropolis method to Hamiltonian Monte Carlo. 2018. arXiv:1706.01520.

[15] Razavian AS, Azizpour H, Sullivan J, Carlsson S. CNN features off-the-shelf: an astounding baseline for recognition. In: 2014 IEEE conference on computer vision and pattern recognition workshops; 2014. p. 512–9. https://doi.org/10.1109/CVPRW.2014.131.

[16] Graves A, Mohamed A, Hinton G. Speech recognition with deep recurrent neural networks. In: 2013 IEEE international conference on acoustics, speech and signal processing; 2013. p. 6645–9. https://doi.org/10.1109/ICASSP.2013.6638947.

[17] Sateesh Babu G, Zhao P, Li X-L. Deep convolutional neural network based regression approach for estimation of remaining useful life. Database systems for advanced applications 2016:214–28. https://doi.org/10.1007/978-3-319-32025-0_14.

[18] Saxena A, Goebel K, Simon D, Eklund N. Damage propagation modeling for aircraft engine run-to-failure simulation. In: 2008 international conference on prognostics and health management; 2008. p. 1–9. https://doi.org/10.1109/PHM.2008.4711414.

[19] Li X, Ding Q, Sun J-Q. Remaining useful life estimation in prognostics using deep convolution neural networks. Reliab Eng Syst Saf 2018;172:1–11.

[20] Srivastava N, Hinton G, Krizhevsky A, Sutskever I, Salakhutdinov R. Dropout: a simple way to prevent neural networks from overfitting. J Mach Learn Res 2014;15(56):1929–58.

[21] Zhang J, Wang P, Yan R, Gao RX. Long short-term memory for machine remaining life prediction. J Manuf Syst 2018;48:78–86. https://doi.org/10.1016/j.jmsy.2018.05.011.

[22] Hochreiter S, Schmidhuber J. Long short-term memory. Neural Comput 1997;9(8):1735–80. https://doi.org/10.1162/neco.1997.9.8.1735.

[23] Ellefsen AL, Bjørlykhaug E, Æsøy V, Ushakov S, Zhang H. Remaining useful life predictions for turbofan engine degradation using semi-supervised deep architecture. Reliab Eng Syst Saf 2019;183:240–51.

[24] Kraus M, Feuerriegel S. Forecasting remaining useful life: interpretable deep learning approach via variational Bayesian inferences. Decis Support Syst 2019;125:113100. https://doi.org/10.1016/j.dss.2019.113100.

[25] Li G, Yang L, Lee C-G, Wang X, Rong M. A Bayesian deep learning RUL framework integrating epistemic and aleatoric uncertainties. IEEE Trans Ind Electron 2020. https://doi.org/10.1109/TIE.2020.3009593.

[26] Peng W, Ye Z-S, Chen N. Bayesian deep-learning-based health prognostics toward prognostics uncertainty. IEEE Trans Ind Electron 2020;67(3):2283–93. https://doi.org/10.1109/TIE.2019.2907440.

[27] Deng Y, Shichang D, Shiyao J, Chen Z, Zhiyuan X. Prognostic study of ball screws by ensemble data-driven particle filters. J Manuf Syst 2020;56:359–72. https://doi.org/10.1016/j.jmsy.2020.06.009.

[28] Chen L, Xu G, Zhang S, Yan W, Wu Q. Health indicator construction of machinery based on end-to-end trainable convolution recurrent neural networks. J Manuf Syst 2020;54:1–11. https://doi.org/10.1016/j.jmsy.2019.11.008.

[29] Zhao R, Yan R, Chen Z, Mao K, Wang P, Gao RX. Deep learning and its applications to machine health monitoring. Mech Syst Signal Process 2019;115:213–37. https://doi.org/10.1016/j.ymssp.2018.05.050.

[30] Deutsch J, He D. Using deep learning-based approach to predict remaining useful life of rotating components. IEEE Trans Syst Man Cybern: Syst 2018;48(1):11–20. https://doi.org/10.1109/TSMC.2017.2697842.

[31] Lampinen J, Vehtari A. Bayesian approach for neural networks – review and case studies. Neural Netw 2001:18. https://doi.org/10.1016/S0893-6080(00)00098-8.

[32] Betancourt M. A conceptual introduction to Hamiltonian Monte Carlo. 2018. arXiv:1701.02434.

[33] Metropolis N, Rosenbluth AW, Rosenbluth MN, Teller AH, Teller E. Equation of state calculations by fast computing machines. J Chem Phys 1953;21(6):1087–92. https://doi.org/10.1063/1.1699114.

[34] Hastings WK. Monte Carlo sampling methods using Markov chains and their applications. Biometrika 1970;57(1):97–109.

[35] Neal RM. MCMC using Hamiltonian dynamics. 2012. arXiv:1206.1901.

[36] Duane S, Kennedy A, Pendleton BJ, Roweth D. Hybrid Monte Carlo. Phys Lett B 1987;195(2):216–22. https://doi.org/10.1016/0370-2693(87)91197-X.

[37] Hoffman MD, Gelman A. The No-U-Turn sampler: adaptively setting path lengths in Hamiltonian Monte Carlo. 2011. arXiv:1111.4246.

[38] Blei DM, Kucukelbir A, McAuliffe JD. Variational inference: a review for statisticians. J Am Stat Assoc 2017;112(518):859–77. https://doi.org/10.1080/01621459.2017.1285773.

[39] Blundell C, Cornebise J, Kavukcuoglu K, Wierstra D. Weight uncertainty in neural networks. 2015. arXiv:1505.05424.

[40] Kingma DP, Ba J. Adam: a method for stochastic optimization. 2017. arXiv:1412.6980.

[41] Kuleshov V, Fenner N, Ermon S. Accurate uncertainties for deep learning using calibrated regression. 2018. arXiv:1807.00263.